# Ingénierie de l'apprentissage automatique

CSI 4506 - automne 2026

Marcel Turcotte
Version: oct. 13, 2026 11h53

# Préambule

## Message du jour




Cohere accroît sa présence au Canada avec un nouveau bureau à Ottawa, 2026-10-08.

> **Aidan Gomez, cofondateur et chef de la direction**
>
> Le Canada est le berceau de Cohere. C'est ici que nous avons fondé l'entreprise, que se trouve une grande partie de nos talents et que nous voulons continuer à bâtir à long terme. Ottawa possède une communauté technologique remarquable et un important bassin de talents. L'ouverture de notre premier bureau dans la ville constitue une étape importante pour investir dans notre équipe et bâtir l'avenir.

Le semestre a commencé par une présentation de Joëlle Pineau et de son rôle chez Cohere. Nous revenons aujourd'hui à cette entreprise alors qu'elle accroît sa présence au Canada en ouvrant un bureau permanent à Ottawa. Ce développement est stimulant

pour la communauté locale de l'IA et pourrait même offrir des possibilités de carrière à certains d'entre vous.

Deux des cofondateurs de Cohere ont des liens intéressants avec Ottawa et la musique.

**Aidan Gomez**, cofondateur et chef de la direction de Cohere, est l'un des huit auteurs de l'article marquant de 2017 *Attention Is All You Need*, qui a introduit l'architecture Transformer à la base des grands modèles de langage modernes. Son parcours en apprentissage automatique possède également un lien avec la musique. Pianiste depuis l'enfance, il a travaillé durant ses études de premier cycle à une application qui utilisait des réseaux de neurones pour suivre une interprétation au piano en direct et tourner automatiquement les pages de la partition.

**Nick Frosst**, un autre cofondateur de Cohere, a grandi à Ottawa et affirme avoir appris la programmation à l'école secondaire Bell, à Bells Corners. Il a ensuite étudié l'informatique et les sciences cognitives à l'Université de Toronto avant de devenir le premier employé du laboratoire Google Brain de Geoffrey Hinton à Toronto. Frosst mène aussi une seconde carrière plutôt inhabituelle : il est le chanteur principal du groupe canadien de rock indépendant **Good Kid**.

## Objectifs d'apprentissage

- **Choisir** un encodage qui respecte la signification d'un attribut et les besoins de l'estimateur.
- **Expliquer** quand la mise à l'échelle importe et **l'appliquer** sans fuite d'information provenant des données de validation ou de test.
- **Comparer** les stratégies de traitement des valeurs manquantes et **effectuer** l'imputation dans un pipeline.
- **Appliquer** les remèdes au déséquilibre des classes aux données d'entraînement sans modifier la distribution d'évaluation.
- **Construire** des attributs non linéaires et des interactions tout en contrôlant la complexité du modèle.
- **Expliquer** comment la qualité, la couverture et l'augmentation des données influencent la généralisation.

# Ingénierie de l'apprentissage automatique

## Ingénierie de l'apprentissage automatique

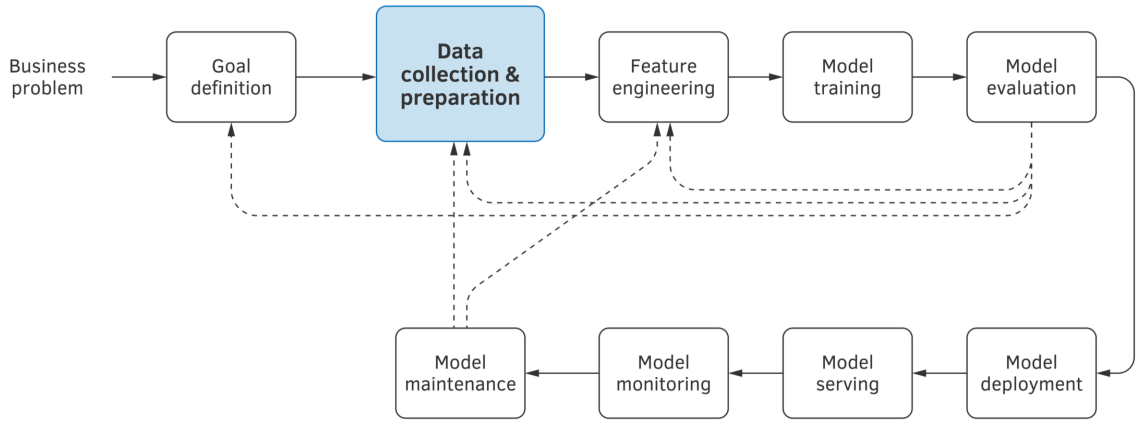



Figure 3.1: Machine learning project life cycle.

**Attribution :** Burkov (2020), figure 3.1

Les cours SDS 3786 : Laboratoire de science des données et CSI 4542 : Fondements de la science des données couvrent en détail l'ingénierie de l'apprentissage automatique. Cette leçon présente un sous-ensemble ciblé des concepts de prétraitement et d'ingénierie des attributs abordés dans ces cours.

Comme l'illustre la figure, le cycle de vie d'un projet d'apprentissage automatique comporte généralement plusieurs itérations. Chaque cycle améliore la compréhension du problème et révèle certaines limites. Une qualité insuffisante des données entraîne souvent l'échec d'un projet.

## Caractéristiques de données de haute qualité

- **Volume** : Le jeu de données est-il assez grand pour soutenir la généralisation?
- **Pertinence** : Les attributs modélisent-ils adéquatement le problème?
- **Représentation** : Les données reflètent-elles les conditions attendues en production?
  - Ces attributs pourront-ils être obtenus à partir des entrées réelles?
  - Le jeu de données couvre-t-il adéquatement les valeurs possibles des attributs?
- **Biais** : Les données sont-elles exemptes de biais?
- **Cohérence des étiquettes** : Les étiquettes sont-elles cohérentes dans tout le jeu de données?

## Cycle d'un projet d'apprentissage automatique

La collecte des données et l'extraction des attributs constituent les deux premières étapes d'un projet d'apprentissage automatique.

1. Recueillir des données **adéquates**.
2. Extraire des **attributs** des données brutes :

- ce processus exige beaucoup de travail;
- il demande de la **créativité**;
- la **connaissance du domaine** est très utile.

# Encodage

## Pourquoi?

Après avoir terminé le devoir 1, vous avez produit un fichier `.csv` contenant des données nettoyées.

Pourquoi d'autres étapes seraient-elles nécessaires?

## Jeu de données Adult

In [2]:
```
import numpy as np
np.random.seed(42)

from sklearn.datasets import fetch_openml

adult = fetch_openml(name='adult', version=2)

print(adult.DESCR)
```

Le jeu de données Adult contient plusieurs attributs à valeurs catégorielles. Il servira de point de départ à une courte présentation de l'encodage de ces valeurs.

## Adult - Workclass

In [3]:
```
adult.data['workclass'].unique()
```

```
['Private', 'Local-gov', NaN, 'Self-emp-not-inc', 'Federal-gov', 'State-go
v', 'Self-emp-inc', 'Without-pay', 'Never-worked']
Categories (8, str): ['Federal-gov', 'Local-gov', 'Never-worked', 'Private',
'Self-emp-inc', 'Self-emp-not-inc', 'State-gov', 'Without-pay']
```

## Adult - Education

In [4]:
```
adult.data['education'].unique()
```

```
['11th', 'HS-grad', 'Assoc-acdm', 'Some-college', '10th', ..., 'Assoc-voc',
'9th', '12th', '1st-4th', 'Preschool']
Length: 16
Categories (16, str): ['10th', '11th', '12th', '1st-4th', ..., 'Masters', 'P
reschool', 'Prof-school', 'Some-college']
```

# Adult - Marital Status

In [5]:
```
adult.data['marital-status'].unique()
```

```
['Never-married', 'Married-civ-spouse', 'Widowed', 'Divorced', 'Separated',
'Married-spouse-absent', 'Married-AF-spouse']
Categories (7, str): ['Divorced', 'Married-AF-spouse', 'Married-civ-spouse',
'Married-spouse-absent', 'Never-married', 'Separated', 'Widowed']
```

# Données catégorielles

**Principes de représentation des données**

- **Représentation numérique** : de nombreux algorithmes d'apprentissage exigent des vecteurs d'attributs numériques.
- **La signification compte** : la représentation doit respecter le caractère non ordonné ou ordonné des catégories.

# Méthodes d'encodage

Considérons l'attribut `workclass`, qui possède huit valeurs distinctes, telles que 'Federal-gov' et 'Local-gov'.

- **Quelle** méthode d'encodage faut-il privilégier et **pourquoi**?

1. `w` = 1, 2, 3, 4, 5, 6, 7 ou 8
2. `w` = [0,0,0], [0,0,1], [0,1,0], . . . ou [1,1,1]
3. `w` = [1,0,0,0,0,0,0,0], [0,1,0,0,0,0,0,0], . . . ou [0,0,0,0,0,0,0,1]

# Encodage des données nominales

L'**encodage one-hot** (*one-hot encoding*) constitue un choix courant pour les catégories non ordonnées, ou **nominales**.

- **Augmentation de la dimension** : l'encodage one-hot crée un attribut de sortie pour chaque catégorie conservée.
- **Absence d'ordre artificiel** : les codes entiers peuvent introduire des distances et un ordre que les catégories ne possèdent pas.
- **Exemple de biais** : la première méthode, `w` = 1, 2, 3, etc., laisse croire que 'Federal-gov' et 'Local-gov' sont semblables, tandis que 'Federal-gov' et 'Without-pay' ne le sont pas.
- **Similarité trompeuse** : la deuxième méthode, `w` = [0,0,0], [0,0,1], etc., peut induire l'algorithme en erreur en suggérant une similarité fondée sur des motifs numériques.

## Définition

L'**encodage one-hot** transforme une variable catégorielle en vecteur binaire. Chaque catégorie est représentée par un vecteur qui contient un seul « 1 » et des « 0 » partout ailleurs.

Les attributs de grande cardinalité peuvent nécessiter d'autres représentations, comme le regroupement des catégories rares, le hachage, l'encodage par la cible ou des plongements appris.

## OneHotEncoder

In [6]:
```
from sklearn.preprocessing import OneHotEncoder

work = adult.data[['workclass']]

onehot_encoder = OneHotEncoder(handle_unknown="ignore")

onehot_encoder.fit(work)
values_encoded = onehot_encoder.transform(work)

values_encoded[:5].toarray()
```

```
array([[0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1.]])
```

**La cohérence est essentielle** : utilisez le même encodage pour les ensembles de validation et de test ainsi que pour les données de production.

Durant `fit`, un encodeur apprend les catégories et l'ordre de leurs colonnes. Ajuster un deuxième encodeur sur les données de validation, de test ou de production peut donc changer la signification ou le nombre des colonnes. Il faut plutôt ajuster l'encodeur sur les données d'entraînement et réutiliser sa méthode `transform` partout ailleurs. Un pipeline applique automatiquement ce principe durant la validation croisée.

`Pandas` offre aussi la méthode `get_dummies()` pour effectuer un encodage one-hot, mais les deux outils présentent des différences importantes :

- **Mémorisation des catégories** : `OneHotEncoder` conserve les catégories observées pendant l'entraînement, contrairement à `get_dummies()`.
- **Cohérence en production** : le schéma d'encodage employé en production doit être identique à celui de l'entraînement.
- **Longueur des vecteurs** : si `get_dummies()` rencontre un nombre différent de catégories dans de nouvelles données, la longueur des vecteurs peut changer et causer des erreurs.

- **Valeurs manquantes** : par défaut, `get_dummies()` encode une valeur manquante par des zéros dans toutes les colonnes. Une colonne explicite est créée seulement avec `dummy_na=True`.

La cohérence de l'encodage entre les données d'**entraînement**, de **validation** et de **production** est essentielle à l'intégrité du modèle.

## Étude de cas

- **Jeu de données** : Heart Disease
  - **Exemples** : 303, **attributs** : 13
  - **Cible** : 0 indique l'absence de maladie; les valeurs 1 à 4 indiquent sa présence
- **Données catégorielles** :
  - **sex** : 1 = homme, 0 = femme
  - **cp (type de douleur thoracique)** :
    - 1 : angine typique
    - 2 : angine atypique
    - 3 : douleur non angineuse
    - 4 : asymptomatique
  - **Autres** : 'fbs', 'restecg', 'exang', 'slope', 'thal'

**Pour simplifier l'analyse** : les exemples qui contiennent des valeurs manquantes sont supprimés, la cible est ramenée à absence ou présence et aucun réglage d'hyperparamètres n'est effectué.

La cible du jeu de données UCI indique l'absence par 0 et différents degrés de présence par les valeurs 1 à 4. Le code convertit ces valeurs en la tâche binaire décrite sur la diapositive.

Les lignes supprimées peuvent différer systématiquement des lignes complètes. Ce raccourci peut donc introduire un biais de sélection. Une analyse réaliste examinerait le mécanisme des valeurs manquantes et comparerait une stratégie d'imputation ajustée sur les données d'entraînement.

L'exemple se limite volontairement au prétraitement. Les choix du modèle et de la régularisation seraient effectués à partir des données de validation ou par validation croisée, tandis que l'ensemble de test serait réservé à l'évaluation finale.

## Étude de cas

In [7]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```python
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.pipeline import Pipeline

# Charger le jeu de données 'Heart-Disease' depuis OpenML
data = fetch_openml(name='Heart-Disease', version=1, as_frame=True)
df = data.frame

# Remplacer '?' par NaN et convertir les colonnes en valeurs numériques
for col in df.columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Supprimer les lignes qui contiennent des valeurs manquantes
df.dropna(inplace=True)

# Définir les attributs et convertir la cible en absence ou présence
X = df.drop(columns=['target'])
y = (df['target'] > 0).astype(int)

# Colonnes à encoder avec OneHotEncoder
columns_to_encode = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'thal'

# Repérer les colonnes numériques
numerical_columns = X.columns.difference(columns_to_encode)

# Diviser les données avant d'appliquer les transformations
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Appliquer OneHotEncoder et StandardScaler avec ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        (
            'onehot',
            OneHotEncoder(drop='first', handle_unknown='ignore'),
            columns_to_encode,
        ),
        ('scaler', StandardScaler(), numerical_columns)
    ]
)

# Le pipeline ajuste le prétraitement et le classificateur sur l'entraînemer
heart_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression()),
])
heart_pipeline.fit(X_train, y_train)
```

`ColumnTransformer` applique différentes transformations aux colonnes choisies. Un transformateur peut aussi être `drop`, qui exclut les colonnes choisies, ou `passthrough`, qui les conserve sans les modifier.

Le `Pipeline` qui l'englobe constitue le principe d'ingénierie essentiel. L'appel à `fit` apprend les paramètres de prétraitement et le classificateur uniquement à partir des données d'entraînement. L'appel à `predict` applique les transformations déjà ajustées avant la classification. Durant la validation croisée, chaque pli apprend donc ses propres paramètres de prétraitement à partir de sa partition d'entraînement.

## Étude de cas - résultats

In [8]:
```
# Prédire et évaluer sur l'ensemble de test resté intact
y_pred = heart_pipeline.predict(X_test)

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.83      0.71      0.77        28
           1       0.78      0.88      0.83        33

    accuracy                           0.80        61
   macro avg       0.81      0.80      0.80        61
weighted avg       0.81      0.80      0.80        61
```

## Étude de cas - douleur thoracique (cp)

In [9]:
```
# Récupérer les noms des attributs et les coefficients du pipeline ajusté
feature_names = heart_pipeline[
    'preprocessor'
].get_feature_names_out()

coefficients = heart_pipeline['classifier'].coef_[0]

# Créer un DataFrame qui associe les attributs et les coefficients
coef_df = pd.DataFrame({
    'Attribut': feature_names,
    'Coefficient': coefficients
})

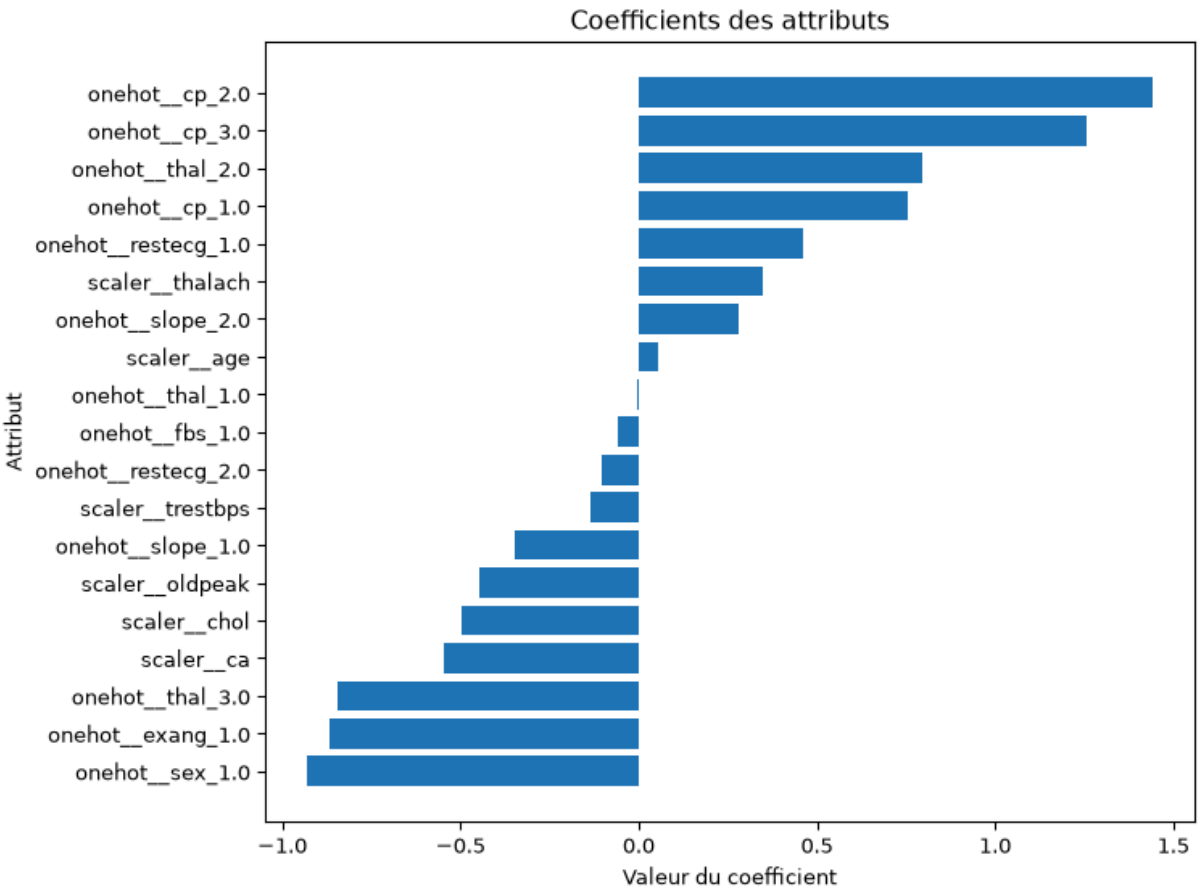
# Afficher les coefficients associés à 'cp'
cp_features = coef_df[coef_df['Attribut'].str.contains('_cp')]
print("\nCoefficients associés à 'cp' :")
print(cp_features)
```

```
Coefficients associés à 'cp' :
         Attribut  Coefficient
1  onehot__cp_1.0     0.754906
2  onehot__cp_2.0     1.440017
3  onehot__cp_3.0     1.255102
```

## Étude de cas - coefficients

In [10]:
```
# Visualiser les coefficients ajustés

coef_plot_df = coef_df.sort_values(by='Coefficient')
plt.figure(figsize=(8, 6))
plt.barh(coef_plot_df['Attribut'], coef_plot_df['Coefficient'])
plt.title('Coefficients des attributs')
plt.xlabel('Valeur du coefficient')
plt.ylabel('Attribut')
plt.tight_layout()
plt.show()
```




Lorsque la première catégorie de chaque attribut encodé sert de référence, un coefficient positif augmente le logarithme du rapport des probabilités ajusté de présence d'une maladie cardiaque par rapport à cette référence, toutes les autres valeurs transformées étant maintenues constantes. Un coefficient négatif le réduit.

L'amplitude d'un coefficient dépend aussi de la mise à l'échelle et de la régularisation. Ces coefficients décrivent des associations dans le modèle ajusté; ils ne démontrent pas qu'un attribut cause une maladie cardiaque.

## Définition

L'**encodage ordinal** attribue des valeurs numériques à des catégories selon leur ordre ou leur rang intrinsèque.

## Ingénierie des attributs - données ordinales

Pour des valeurs comme **« faible »**, **« moyen »** et **« bon »**, un encodage ordinal peut préserver l'ordre.

. . .

Cependant!

In [11]:
```
from sklearn.preprocessing import OrdinalEncoder

X =[['Faible'], ['Moyen'], ['Bon'], ['Moyen'], ['Moyen']]

encoder = OrdinalEncoder()

encoder.fit(X)
encoder.transform(X)
```

```
array([[1.],
       [2.],
       [0.],
       [2.],
       [2.]])
```

## OrdinalEncoder (révisé)

In [12]:
```
from sklearn.preprocessing import OrdinalEncoder

X =[['Faible'], ['Moyen'], ['Bon'], ['Moyen'], ['Moyen']]

encoder = OrdinalEncoder(categories=[['Faible', 'Moyen', 'Bon']])

encoder.fit(X)

X_encoded = encoder.transform(X)

X_encoded
```

```
array([[0.],
       [1.],
       [2.],
       [1.],
       [1.]])
```

L'ordre voulu des catégories doit être fourni explicitement à l'encodeur; autrement, celui-ci utilise l'ordre alphabétique.

Un encodeur ordinal convient lorsque les valeurs catégorielles possèdent un ordre ou un rang intrinsèque clair, comme « faible », « moyen » et « élevé », ou « médiocre », « moyen » et « bon ». Cette méthode préserve les relations ordinales entre les catégories.

Lorsque les données sont véritablement ordinales, cet encodage compact peut avantager certains modèles. Si leur nature ordinale est incertaine, il est plus prudent d'utiliser `OneHotEncoder`.

## Définition

La **discrétisation** transforme un attribut numérique continu ou finement gradué en un ensemble fini d'intervalles.

Aussi appelée **regroupement en intervalles** (*binning* ou *bucketing*) ou **quantification** (*quantization*).

## Ingénierie des attributs : discrétisation

**Exemple** : regrouper les âges en catégories comme « nourrisson », « enfant », « adolescent », « adulte » et « personne âgée ».

. . .

**Avantage possible** :

- encode des seuils pertinents du domaine et peut réduire la complexité du modèle.

**Inconvénients** :

- élimine les différences à l'intérieur de chaque intervalle et introduit des frontières abruptes;
- exige des seuils justifiables. Par exemple, la catégorie « personne âgée » pourrait commencer à 60, 65 ou 70 ans[1].

Des intervalles prédéfinis peuvent aider un arbre de décision à produire un modèle plus compact, car l'arbre n'a plus à découvrir chaque seuil à partir de l'échantillon disponible.

Ce biais inductif prononcé peut toutefois masquer des variations utiles ou empêcher l'estimateur de découvrir de meilleures frontières. La discrétisation constitue donc un choix de modélisation, et non une étape automatique du prétraitement.

La validation croisée permet de comparer des schémas d'encodage. L'ensemble de test doit néanmoins rester à l'écart jusqu'à l'évaluation finale afin de prévenir une fuite de données et de préserver l'impartialité de l'évaluation.

## FunctionTransformer

[1] Votre professeur s'intéresse à votre choix de seuil

In [13]:
```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import FunctionTransformer

bins = [0, 1, 13, 20, 60, np.inf]
labels = ['nourrisson', 'enfant', 'adolescent', 'adulte', 'personne âgée']

transformer = FunctionTransformer(
    pd.cut, kw_args={'bins': bins, 'labels': labels, 'retbins': False}
)

X = np.array([0.5, 2, 15, 25, 97])
transformer.fit_transform(X)
```

```
['nourrisson', 'enfant', 'adolescent', 'adulte', 'personne âgée']
Categories (5, str): ['nourrisson' < 'enfant' < 'adolescent' < 'adulte' < 'p
ersonne âgée']
```

Voir aussi : KBinsDiscretizer

# Mise à l'échelle

## Qu'est-ce que c'est?

La **mise à l'échelle des attributs** (*feature scaling*) place leurs valeurs dans des **plages comparables**.

## Pourquoi?

Pourquoi certains estimateurs exigent-ils des attributs d'échelles comparables?

La mise à l'échelle modifie la géométrie et le conditionnement numérique d'un problème d'apprentissage. Son utilité dépend de l'estimateur.

Les méthodes fondées sur les distances, comme les $k$ plus proches voisins et les $k$ moyennes, y sont sensibles puisqu'un attribut à grande échelle peut dominer la distance. Les noyaux RBF et les modèles linéaires régularisés sont aussi sensibles à l'échelle. L'optimisation par gradient converge souvent plus facilement lorsque les attributs ont des échelles comparables.

Les modèles à base d'arbres comparent des seuils à l'intérieur d'un attribut à la fois. Une remise à l'échelle monotone ne change donc généralement pas les partitions possibles.

La mise à l'échelle ne décide pas de l'importance qu'un attribut *devrait* avoir. La connaissance du domaine et la géométrie recherchée pour le modèle demeurent importantes.

## Scénario

Nous voulons prédire le prix d'une maison par **régression des $k$ plus proches voisins (KNN)** à partir de deux attributs :

- $x_1$ : nombre de pièces (petite échelle)
- $x_2$ : superficie en pieds carrés (grande échelle)

## Données (trois maisons)

In [14]:
```
import numpy as np
import pandas as pd

# Trois exemples (pièces, pieds carrés); prix pour b et c seulement
point_names = ["a", "b", "c"]
X = np.array([
    [4, 1500.0],  # a (requête)
    [8, 1520.0],  # b (entraînement)
    [4, 1300.0],  # c (entraînement)
], dtype=float)

prices = pd.Series([np.nan, 520_000, 390_000], index=point_names, name="prix

df = pd.DataFrame(X, columns=["pièces", "pieds_carrés"], index=point_names)
display(df)
display(prices.to_frame())
```

**Remarque.** Les exemples **b** et **c** forment l'ensemble d'entraînement. L'exemple **a** constitue la requête dont nous voulons prédire le prix.

## Distances euclidiennes (sans mise à l'échelle)

Lorsqu'un attribut possède une échelle beaucoup plus grande, comme la superficie, il peut dominer la somme.

In [15]:
```
from sklearn.metrics import pairwise_distances

dist_unscaled = pd.DataFrame(
    pairwise_distances(df.values, metric="euclidean"),
    index=df.index, columns=df.index
)
dist_unscaled
```

In [16]:
```
print("Plus proche de 'a' (sans mise à l'échelle) :", dist_unscaled.loc["a"]
```

```
Plus proche de 'a' (sans mise à l'échelle) : b
```

## Mise à l'échelle appropriée

Pour éviter toute fuite de données, calculons les paramètres de mise à l'échelle à partir des seules données d'entraînement (**b** et **c**), puis transformons les données d'entraînement et la requête :

In [17]:
```
from sklearn.preprocessing import StandardScaler

train_idx = ["b", "c"]
query_idx = ["a"]

scaler = StandardScaler()

scaler.fit(df.loc[train_idx])     # ajuster uniquement sur l'entraînement

Z = pd.DataFrame(
    scaler.transform(df),
    columns=df.columns, index=df.index
)

Z
```

## Distances euclidiennes (après mise à l'échelle)

In [18]:
```
dist_scaled = pd.DataFrame(
    pairwise_distances(Z.values, metric="euclidean"),
    index=Z.index, columns=Z.index
)
dist_scaled
```

In [19]:
```
print("Plus proche de 'a' (après mise à l'échelle) :", dist_scaled.loc["a"].
```

```
Plus proche de 'a' (après mise à l'échelle) : c
```

## Régression KNN

Un régresseur 1-NN prédit le prix de l'exemple d'entraînement le plus proche. Comparons le même estimateur avec et sans mise à l'échelle.

In [20]:
```
from sklearn.neighbors import KNeighborsRegressor

X_train = df.loc[train_idx].values
y_train = prices.loc[train_idx].values
X_query = df.loc[query_idx].values

knn_plain = KNeighborsRegressor(n_neighbors=1, metric="euclidean")
knn_plain.fit(X_train, y_train)

knn_scaled = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=1, metric="euclidean"))
])
```

```
knn_scaled.fit(X_train, y_train)
```

Le pipeline se comporte comme un seul régresseur. Sa méthode `fit` apprend les paramètres de mise à l'échelle à partir de `X_train`, puis ajuste KNN dans l'espace transformé. Sa méthode `predict` applique cette transformation déjà ajustée avant de chercher les voisins.

## Régression KNN : résultats

In [21]:
```
pred_plain = knn_plain.predict(X_query)[0]
pred_scaled = knn_scaled.predict(X_query)[0]

plain_index = knn_plain.kneighbors(
    X_query, return_distance=False
)[0, 0]
scaled_query = knn_scaled["scaler"].transform(X_query)
scaled_index = knn_scaled["knn"].kneighbors(
    scaled_query, return_distance=False
)[0, 0]

knn_comparison = pd.DataFrame(
    {
        "prédiction": [pred_plain, pred_scaled],
        "plus proche voisin": [
            train_idx[plain_index], train_idx[scaled_index]
        ],
    },
    index=["sans mise à l'échelle", "avec mise à l'échelle"],
)
```

| | Prédiction | Plus proche voisin |
|---|---|---|
| **Sans mise à l'échelle** | 520 000 $ | b |
| **Avec mise à l'échelle** | 390 000 $ | c |

Ce jeu de données volontairement minuscule illustre la géométrie, et non la qualité du modèle. Deux observations ne suffisent pas à estimer de façon stable les moyennes et écarts-types de la population. De plus, le prix de la requête est inconnu et ne permet pas de mesurer la performance prédictive. L'exemple montre un changement du voisin et de la prédiction, et non une amélioration de la performance.

## Mise à l'échelle min-max

La mise à l'échelle min-max transforme chaque attribut dans un intervalle choisi, habituellement $[0, 1]$.

$$\frac{x_i^{(j)} - \min^{(j)}}{\max^{(j)} - \min^{(j)}}$$

Le minimum et le maximum doivent être appris à partir des données d'entraînement.

**Voir :** sklearn.preprocessing.MinMaxScaler

## Standardisation

La standardisation soustrait la moyenne d'entraînement et divise par l'écart-type d'entraînement.

$$\frac{x_i^{(j)} - \mu^{(j)}}{\sigma^{(j)}}$$

L'attribut d'entraînement transformé possède une moyenne de 0 et une variance de 1, mais sa distribution ne devient **pas** normale et son étendue n'est pas bornée.

**Voir :** sklearn.preprocessing.StandardScaler

## Choisir une méthode de mise à l'échelle

- La **standardisation** constitue un choix courant pour les méthodes fondées sur les distances, les modèles linéaires régularisés et l'optimisation par gradient.
- La **mise à l'échelle min-max** convient lorsqu'une plage d'entrée bornée est exigée.
- Les deux méthodes sont sensibles aux valeurs aberrantes : la mise à l'échelle min-max utilise les extrêmes, tandis que la standardisation utilise la moyenne et l'écart-type.
- Pour les attributs très asymétriques ou qui contiennent des valeurs aberrantes, examinez le processus de production des données et envisagez une transformation justifiée, une règle d'écrêtage ou `RobustScaler`.
- Lorsque plusieurs choix sont plausibles et importants, comparez-les au moyen de la validation croisée.

La mise à l'échelle et la transformation d'une distribution résolvent des problèmes différents. Un logarithme peut réduire l'asymétrie à droite de données strictement positives, mais il doit avoir un sens pour l'attribut et faire partie du même flux de travail sans fuite de données. `RobustScaler` utilise la médiane et l'écart interquartile, ce qui rend ses statistiques ajustées moins sensibles aux valeurs extrêmes.

## Comparaison des méthodes

| Propriété | Standardisation | Mise à l'échelle min-max |
|---|---|---|
| **Plage de sortie** | Non bornée | Habituellement [0, 1] |
| **Centre** | Moyenne à 0 | Non centrée |
| **Sensibilité aux valeurs aberrantes** | Par la moyenne et l'écart-type | Par le minimum et le maximum |
| **Usage typique** | Variances comparables et attributs centrés à zéro | Plage bornée prescrite |

# Éviter les fuites de données!

Le **flux de travail correct** :

- diviser les exemples avant d'apprendre tout paramètre de prétraitement;
- ajuster l'outil de mise à l'échelle sur les **données d'entraînement**;
- réutiliser cet outil ajusté pour transformer les données de validation, de test et de production;
- pendant la validation croisée, ajuster le prétraitement séparément dans chaque pli d'entraînement, normalement à l'aide d'un `Pipeline`.

Les moyennes, écarts-types, minima, maxima, ensembles de catégories et valeurs d'imputation constituent tous de l'information apprise. Ils doivent provenir uniquement des données d'entraînement.

L'ensemble de test simule des données jamais observées. S'il influence le prétraitement, la procédure d'entraînement reçoit de l'information qui ne serait pas disponible pour une nouvelle prédiction. Cette **fuite de données** (*data leakage*) peut produire une estimation trop optimiste de la capacité de généralisation.

# Échantillon normal avec valeurs aberrantes

In [22]:
```
import numpy as np
np.random.seed(7)

# Caractéristiques de l'échantillon
sample_size = 1000
mu = 57
sigma = 7

# Générer les valeurs
norm_values = sigma * np.random.randn(sample_size) + mu

# Ajouter trois valeurs aberrantes
norm_values = np.append(norm_values, [92, 95, 98])
```

## Distribution originale

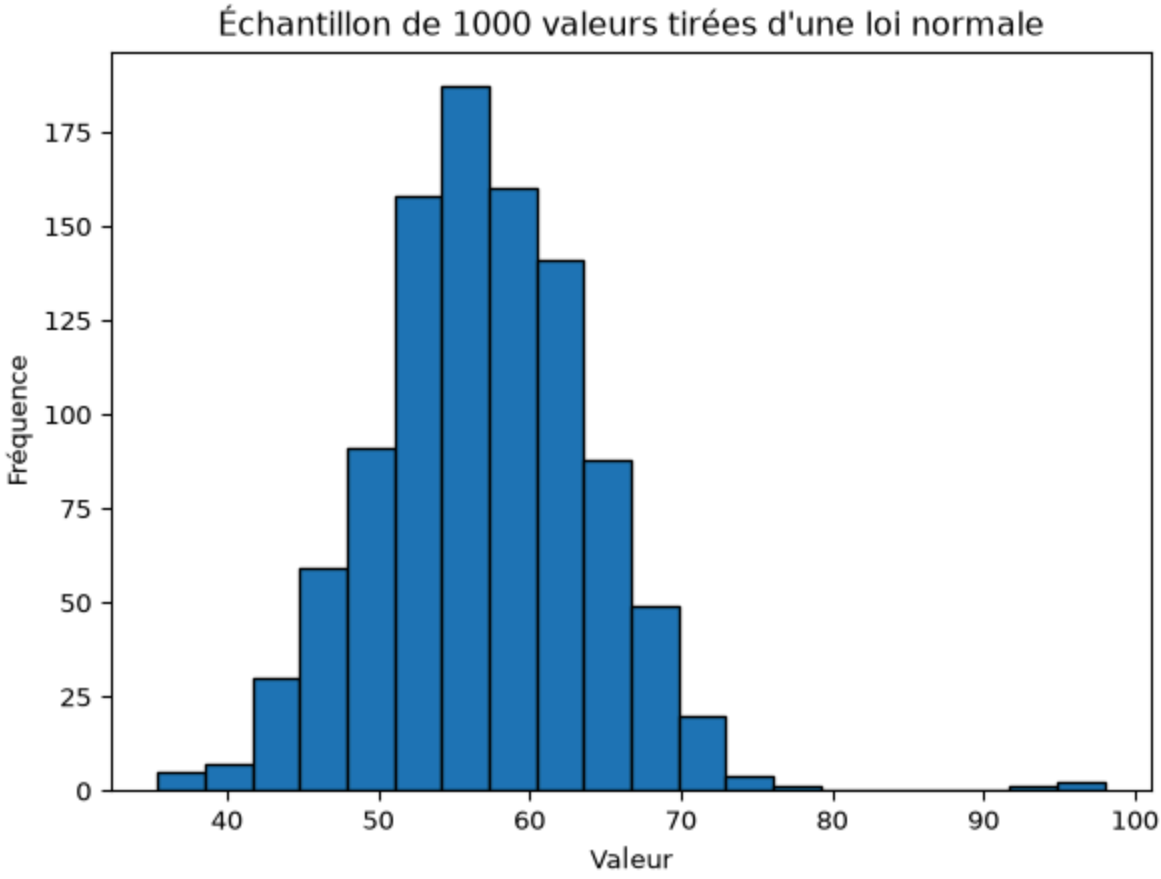
In [23]:
```
import matplotlib.pyplot as plt

# Tracer l'histogramme
plt.hist(norm_values, bins=20, edgecolor='black')

plt.title(f'Échantillon de {sample_size} valeurs tirées d\'une loi normale')

plt.xlabel('Valeur')
plt.ylabel('Fréquence')

plt.show()
```




## Mise à l'échelle min-max

In [24]:
```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

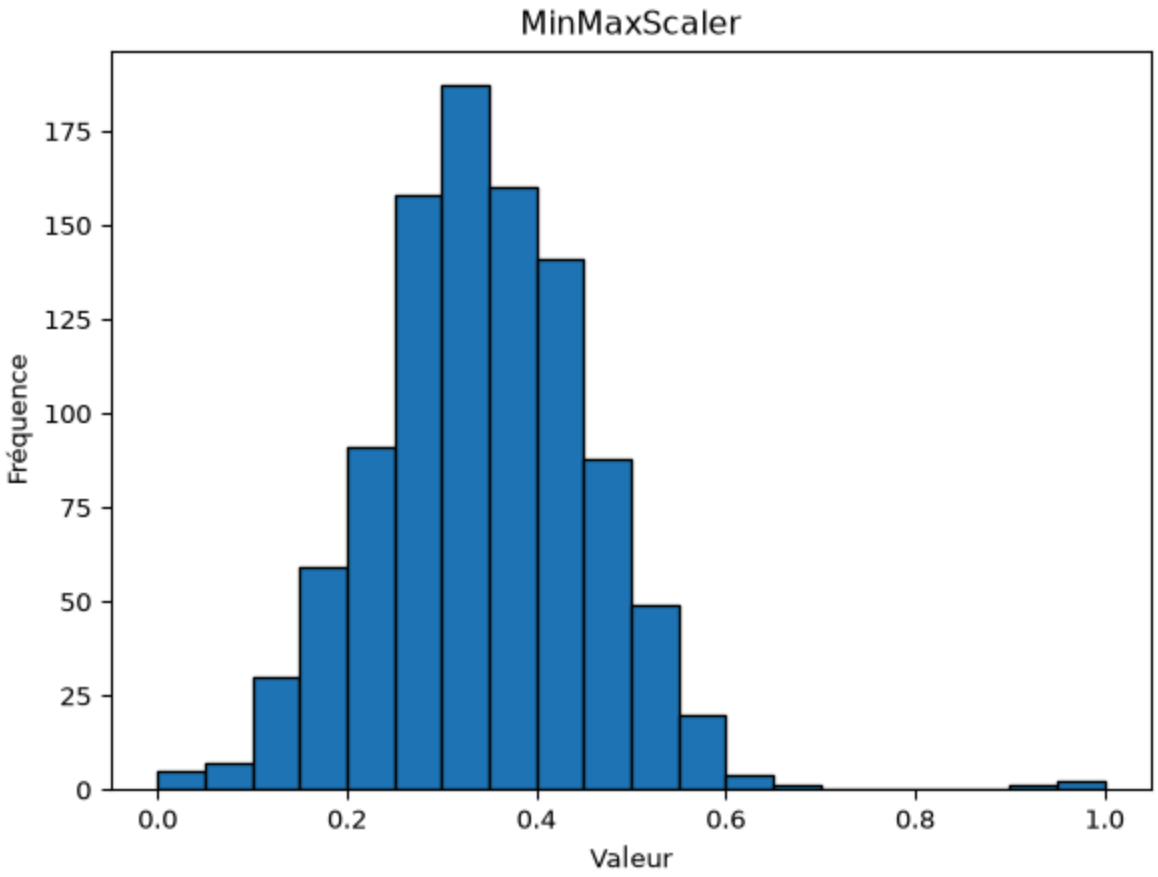
minmax_norm_values = scaler.fit_transform(norm_values.reshape(-1, 1))

# Tracer l'histogramme
plt.hist(minmax_norm_values, bins=20, edgecolor='black')
```

```
plt.title('MinMaxScaler')

plt.xlabel('Valeur')
plt.ylabel('Fréquence')

plt.show()
```




Mise à l'échelle min-max de valeurs provenant d'une loi normale qui contient des valeurs aberrantes élevées.

La plus grande valeur observée est ramenée à 1 et la plus petite à 0. Les valeurs aberrantes élevées étirent cet intervalle et compriment la majorité des observations dans sa partie inférieure. Une mise à l'échelle min-max n'implique pas que la moyenne transformée soit égale à 0,5.

## Standardisation

In [25]:
```
standard_scaler = StandardScaler()

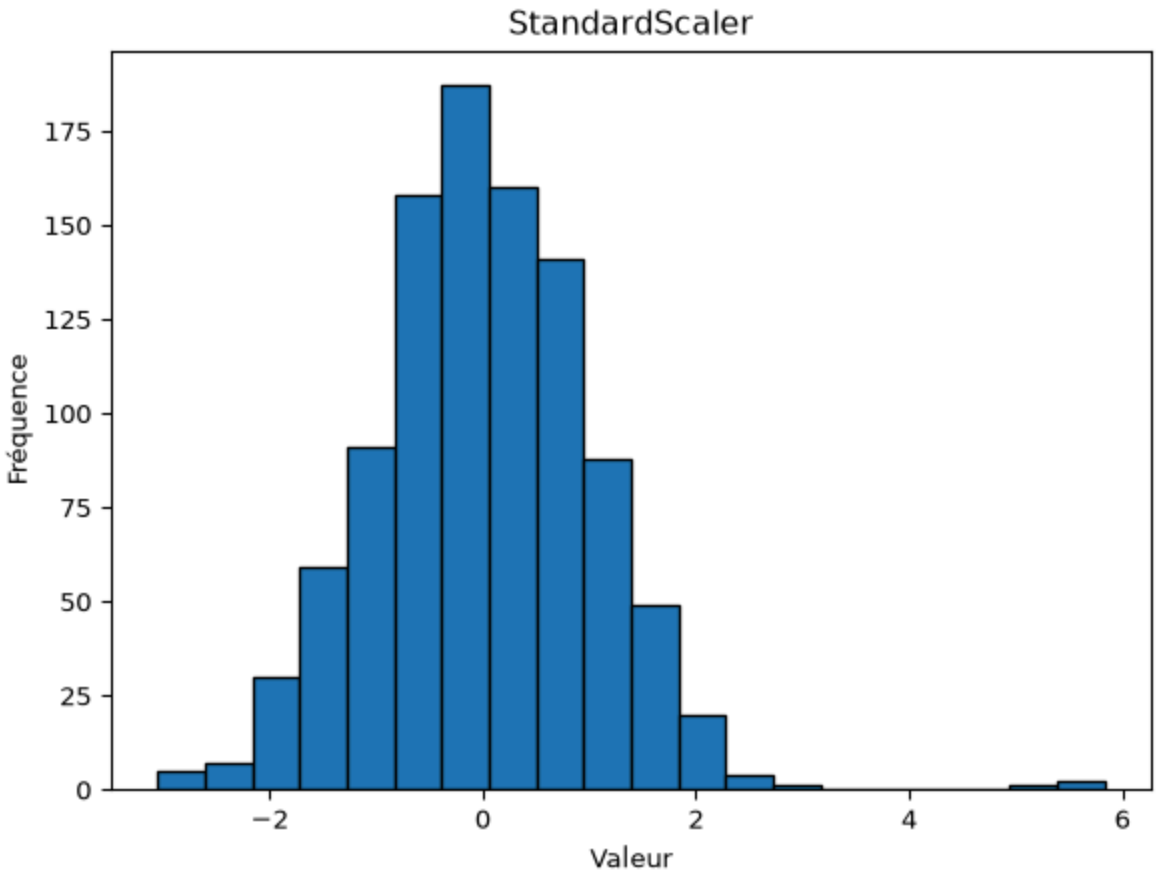
standard_norm_values = standard_scaler.fit_transform(
    norm_values.reshape(-1, 1)
)

# Tracer l'histogramme
plt.hist(standard_norm_values, bins=20, edgecolor='black')
```

```
plt.title('StandardScaler')

plt.xlabel('Valeur')
plt.ylabel('Fréquence')

plt.show()
```




Standardisation ( `StandardScaler` ) de valeurs provenant d'une loi normale qui contient des valeurs aberrantes.

La standardisation change le centre et l'échelle, mais préserve la forme de la distribution. La majorité de cet échantillon approximativement normal se trouve près de zéro, tandis que les valeurs aberrantes demeurent aberrantes. L'intervalle $[-2, 2]$ constitue une description empirique utile de cet échantillon, et non une garantie de `StandardScaler` .

# Valeurs manquantes

## Définition

Les **valeurs manquantes** désignent l'absence d'une donnée à un endroit où une valeur est attendue dans un jeu de données.

L'**âge** en constitue un bon exemple, puisque certaines personnes peuvent refuser de le divulguer pour des raisons de confidentialité.

## Traiter les valeurs manquantes

- **Supprimer des exemples**
  - Ce choix est raisonnable seulement s'il reste assez de données et si les lignes incomplètes ne diffèrent pas systématiquement des autres.
- **Supprimer des attributs**
  - Ce choix convient lorsqu'un attribut n'est pas essentiel et qu'il reste trop peu d'information fiable.
- **Employer un algorithme qui accepte les valeurs manquantes**
  - Certains estimateurs les modélisent directement; beaucoup d'autres ne les acceptent pas.
- **Effectuer une imputation**
  - Remplacer les valeurs manquantes à l'aide de statistiques ou d'un modèle appris sur les données d'entraînement.

## Définition

L'**imputation des données** consiste à remplacer les valeurs manquantes par des valeurs de substitution, généralement calculées à l'aide de méthodes statistiques ou d'apprentissage automatique.

## Stratégie d'imputation

Remplaçons une valeur numérique manquante par la **moyenne** ou la **médiane** d'entraînement de l'attribut.

In [26]:
```
from sklearn.impute import SimpleImputer

X_train_missing = np.array([
    [1.0, 10.0],
    [2.0, np.nan],
    [3.0, 12.0],
])
X_new_missing = np.array([[4.0, np.nan]])

imputer = SimpleImputer(strategy="median")
imputer.fit(X_train_missing)
imputer.transform(X_new_missing)
```

```
array([[ 4., 11.]])
```

...

- **Limite** : l'imputation univariée ignore les relations entre les attributs et réduit la variabilité observée.
- **Attributs catégoriels** : utilisez la catégorie la plus fréquente ou une catégorie explicite `"manquant"` lorsque cette représentation convient.

Les données peuvent être manquantes **complètement au hasard**, en fonction d'autres attributs observés ou en fonction de la valeur manquante elle-même. On décrit souvent ces mécanismes par les sigles MCAR, MAR et MNAR. Un outil d'imputation simple ne détermine pas quel mécanisme a produit les valeurs manquantes.

L'imputation par la moyenne, la médiane ou la valeur la plus fréquente fournit des points de comparaison utiles, mais elle peut déformer les distributions et les relations. La stratégie d'imputation fait partie du modèle et doit être évaluée sur des données tenues à l'écart.

Comme pour la mise à l'échelle, l'outil d'imputation doit être ajusté uniquement sur la partition d'entraînement. Un pipeline répète correctement cet ajustement dans chaque pli de validation croisée.

## Préserver l'information de valeurs manquantes

Une valeur imputée peut être accompagnée d'un attribut binaire qui indique que la valeur originale était manquante.

In [27]:
```
imputer_with_indicator = SimpleImputer(
    strategy="median", add_indicator=True
)
imputer_with_indicator.fit_transform(X_train_missing)
```

```
array([[ 1., 10.,  0.],
       [ 2., 11.,  1.],
       [ 3., 12.,  0.]])
```

`add_indicator=True` permet à un estimateur d'utiliser comme signal distinct le fait qu'une valeur était manquante. Cette information peut aider lorsque le processus d'observation est informatif, par exemple lorsqu'une mesure particulière est demandée seulement pour les personnes qui présentent un risque plus élevé.

Pour certains modèles à base d'arbres, une valeur sentinelle située hors de la plage normale peut jouer un rôle similaire. Elle peut cependant déformer fortement les distances ou les relations linéaires. Un indicateur explicite communique plus clairement l'intention.

## Prédire un attribut manquant

- Choisir comme cible de prédiction un attribut qui contient des valeurs manquantes.
- Utiliser comme exemples d'entraînement les lignes où cet attribut est observé.
- Utiliser les autres attributs observés comme entrées.
- Prédire les valeurs manquantes au moyen d'un modèle de régression ou de classification.

## Apprentissage automatique et imputation

1. **Méthode fondée sur les voisins** :
    - employer les $k$ plus proches voisins pour repérer les $k$ exemples les plus semblables et imputer à partir des valeurs non manquantes dans ce voisinage.
2. **Méthodes itératives fondées sur des modèles** :
    - prédire chaque attribut incomplet à partir des autres, puis répéter le processus pour les différents attributs.

Consultez le guide d'imputation de scikit-learn ou la documentation de `KNNImputer`.

## Pourquoi utiliser ces méthodes?

- **Avantage possible** :
    - exploiter les relations entre les attributs que l'imputation univariée ignore.
- **Limites** :
    - l'augmentation du calcul et du nombre de choix de modélisation ne garantit pas de meilleures prédictions;
    - l'outil d'imputation doit tout de même être ajusté dans chaque partition d'entraînement ou pli de validation croisée.

# Remèdes au déséquilibre des classes

## De l'évaluation à l'intervention

La leçon 7 a montré que le déséquilibre des classes peut masquer une mauvaise performance sur une classe peu fréquente et que l'évaluation doit refléter le problème de décision.

Cette leçon pose une question différente :

> Comment la **procédure d'entraînement** peut-elle répondre au déséquilibre des classes sans contaminer l'évaluation?

La leçon 7 a déjà présenté la matrice de confusion, l'exactitude, la précision, le rappel, le score $F_1$, les moyennes micro et macro, les compromis liés au seuil, les courbes ROC et les courbes précision-rappel. Ces définitions et ces formules ne sont pas répétées ici.

Le passage important consiste à quitter la **détection** d'un problème durant l'évaluation pour **modifier** la procédure d'entraînement. Un remède est utile seulement s'il améliore la mesure de validation pertinente pour l'application établie à la leçon 7.

## Remèdes durant l'entraînement

- **Poids de classes ou d'exemples :** faire contribuer davantage les erreurs commises sur certains exemples à l'objectif d'entraînement.
- **Suréchantillonnage aléatoire :** répéter des exemples d'entraînement de la classe minoritaire.
- **Sous-échantillonnage aléatoire :** conserver moins d'exemples d'entraînement de la classe majoritaire.
- **SMOTE :** interpoler des exemples synthétiques de la classe minoritaire à partir d'exemples d'entraînement voisins.
- **Seuil de décision :** choisir un point de fonctionnement à l'aide des données de validation, comme à la leçon 7.

Implémentations : `class_weight` et `imbalanced-learn`.

Les poids de classes modifient la contribution des exemples d'entraînement sans créer ni supprimer d'observations. Le suréchantillonnage peut surajuster les exemples répétés, tandis que le sous-échantillonnage peut éliminer une variation utile de la classe majoritaire.

SMOTE construit des points synthétiques entre des voisins de la classe minoritaire (Chawla et al. 2002). Cette interpolation peut ne pas convenir lorsqu'une ligne droite entre deux observations quitte l'ensemble des exemples plausibles. La version ordinaire de SMOTE suppose aussi des attributs numériques. Les données qui combinent des attributs numériques et catégoriels exigent une variante appropriée, comme `SMOTENC`.

Aucun remède ne convient universellement. Les choix plausibles doivent être comparés avec la procédure de validation et les mesures pertinentes pour l'application établies à la leçon 7.

## Garder l'évaluation intacte

- Préserver la distribution naturelle des classes dans les données de validation et de test.
- Appliquer le rééchantillonnage **uniquement à une partition d'entraînement**.

- Durant la validation croisée, rééchantillonner séparément dans chaque pli d'entraînement.
- Placer le rééchantillonnage et l'estimateur dans un pipeline d'`imbalanced-learn`.
- Réserver l'ensemble de test à l'évaluation finale, après avoir choisi le remède et tout seuil de décision.

Un rééchantillonnage effectué avant la division en ensembles d'entraînement et de test produit une fuite d'information et une distribution d'évaluation irréaliste. Le rééchantillonnage avant la validation croisée pose le même problème, car un exemple synthétique ou répété peut alors influencer les plis d'entraînement et de validation.

Un pipeline d'`imbalanced-learn` appelle l'outil de rééchantillonnage uniquement pendant l'ajustement de chaque pli d'entraînement. Son chemin de prédiction ne rééchantillonne pas les exemples de validation, de test ou de production.

# Nouveaux attributs

## Définition

L'**ingénierie des attributs** (*feature engineering*) consiste à **créer**, à **transformer** et à **sélectionner** des variables à partir des données brutes afin d'améliorer la performance des modèles d'apprentissage automatique.

Nous nous concentrons ici sur la **création** de nouveaux attributs à partir des données brutes.

## Exploration

In [28]:
```
import numpy as np

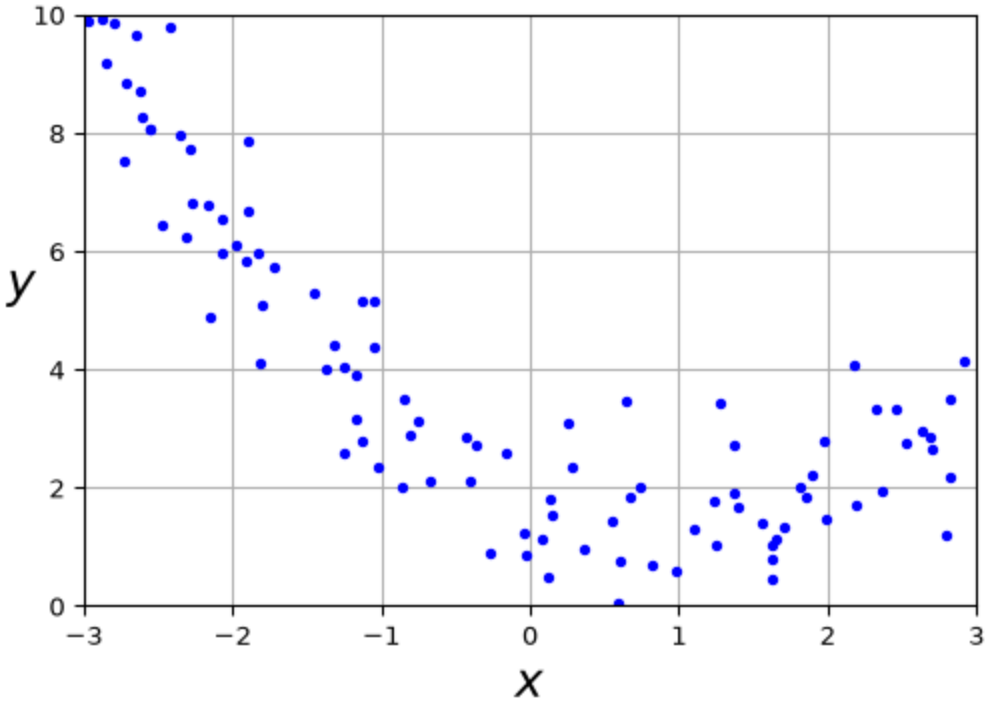
np.random.seed(42)

X = 6 * np.random.rand(100, 1) - 3
y = 0.5 * X ** 2 - X + 2 + np.random.randn(100, 1)

import matplotlib as mpl
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))

plt.plot(X, y, "b.")
plt.xlabel("$x$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.axis([-3, 3, 0, 10])
plt.grid(True)
plt.show()
```




**Attribution :** Géron (2022), chapitre 4

Le nuage de points présente une tendance courbe : la réponse diminue, puis augmente à mesure que $x$ change. Un modèle qui utilise seulement une constante et l'attribut original $x$ ne peut pas représenter cette courbure.

La graine aléatoire fixe rend l'échantillon produit reproductible. Cette reproductibilité aide à comparer les modèles et à déboguer le code, mais une seule graine ne démontre pas qu'un résultat serait stable pour tous les échantillons possibles.

## Régression linéaire

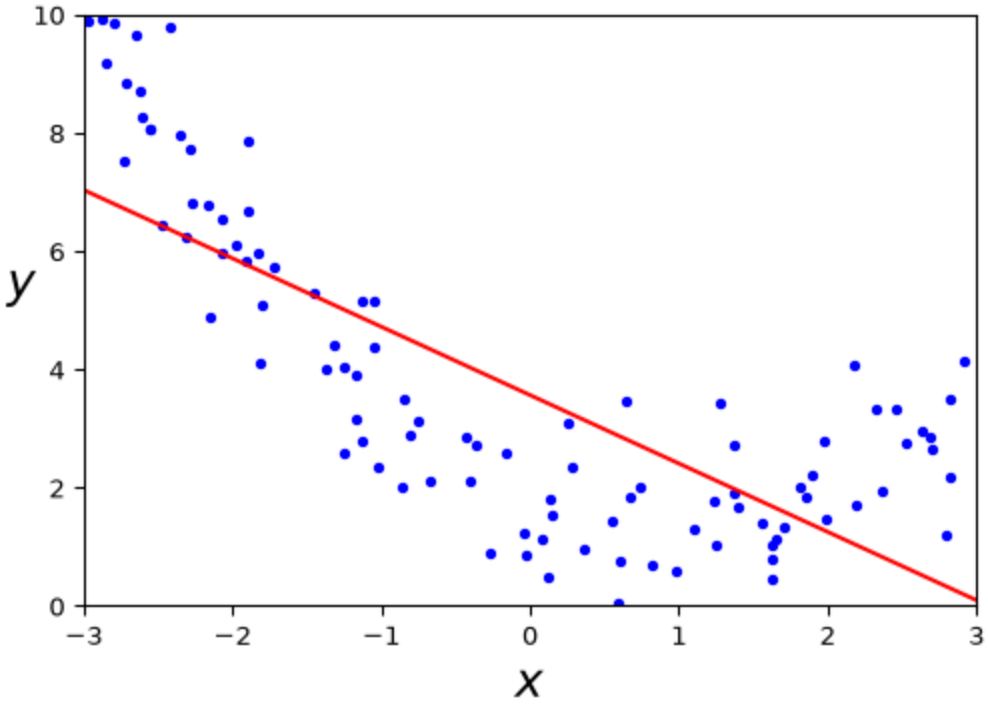
In [29]:
```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X, y)

X_new = np.array([[-3], [3]])
y_pred = lin_reg.predict(X_new)

plt.figure(figsize=(6,4))

plt.plot(X, y, "b.")
plt.plot(X_new, y_pred, "r-")
plt.xlabel("$x$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.axis([-3, 3, 0, 10])
plt.show()
```




Un **modèle linéaire** représente mal ce jeu de données.

La droite la mieux ajustée équilibre les résidus dans le jeu de données. Cependant, modifier sa pente et son ordonnée à l'origine ne peut pas créer de point de retournement. Le modèle sous-ajuste les données parce que sa représentation est trop restrictive pour la relation observée.

## `PolynomialFeatures`

In [30]:
```
from sklearn.preprocessing import PolynomialFeatures

poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)
```

...

In [31]:
```
X[0]
```

```
array([-0.75275929])
```

...

In [32]:
```
X_poly[0]
```

```
array([-0.75275929,  0.56664654])
```

...

> **sklearn.preprocessing.PolynomialFeatures**

> Produit une nouvelle matrice d'attributs composée de toutes les combinaisons polynomiales dont le degré est inférieur ou égal au degré indiqué. Par exemple, pour un échantillon d'entrée bidimensionnel de la forme $[a, b]$, les attributs polynomiaux de degré 2 sont $[1, a, b, a^2, ab, b^2]$.

## `PolynomialFeatures`

Avec deux attributs $a$ et $b$, `PolynomialFeatures` et `degree=3` ajoutent $a^2$, $a^3$, $b^2$, $b^3$, ainsi que $ab$, $a^2b$ et $ab^2$.

. . .

> **Warning**
>
> Avec `include_bias=True`, le nombre total d'attributs de sortie est
>
> $$\binom{D+d}{d} = \frac{(D+d)!}{d!D!},$$
>
> où $D$ est le nombre d'attributs d'entrée. Avec `include_bias=False`, il faut en soustraire un.

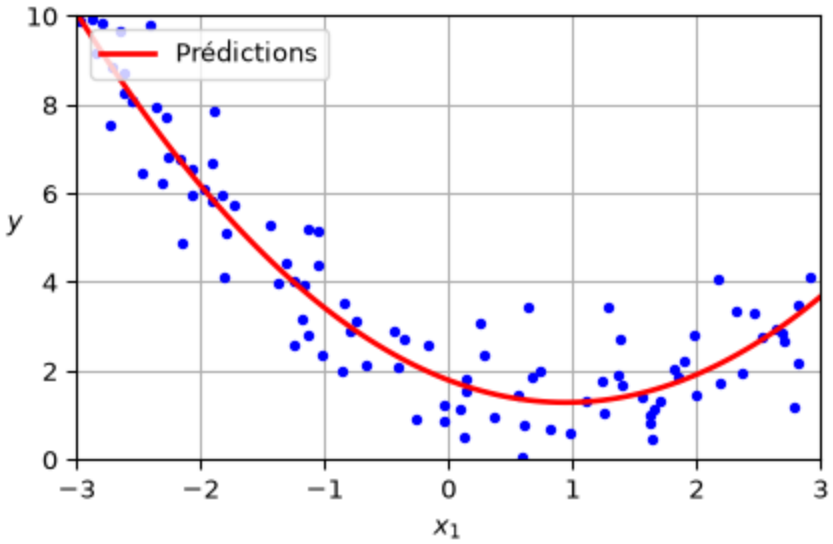
Ce nombre comprend les attributs originaux, leurs puissances et leurs interactions. Il croît rapidement avec $D$ et $d$, ce qui augmente le calcul et le risque de surajustement. Le degré doit être choisi à l'aide des données de validation et la régularisation doit être utilisée lorsqu'elle convient.

## Régression polynomiale

In [33]:
```
lin_reg = LinearRegression()
lin_reg = lin_reg.fit(X_poly, y)

X_new = np.linspace(-3, 3, 100).reshape(100, 1)
X_new_poly = poly_features.transform(X_new)
y_new = lin_reg.predict(X_new_poly)

plt.figure(figsize=(5, 3))
plt.plot(X, y, "b.")
plt.plot(X_new, y_new, "r-", linewidth=2, label="Prédictions")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.legend(loc="upper left")
plt.axis([-3, 3, 0, 10])
plt.grid()
plt.show()
```




`LinearRegression` ajustée aux attributs $[x, x^2]$.

## Polynôme appris

Les données ont été produites par l'équation suivante, à laquelle un bruit gaussien a été ajouté.

$$y = 0.5x^2 - 1.0x + 2.0$$

Voici le modèle appris.

$$\hat{y} = 0.56x^2 + (-1.06)x + 1.78$$

In [34]:
```
lin_reg.coef_, lin_reg.intercept_
```

```
(array([[-1.06633107,  0.56456263]]), array([1.78134581]))
```

La linéarité concerne les coefficients ajustés, et pas nécessairement l'entrée originale. Après avoir défini $z_1 = x$ et $z_2 = x^2$, la régression linéaire ajuste

$$\hat{y} = \theta_0 + \theta_1 z_1 + \theta_2 z_2.$$

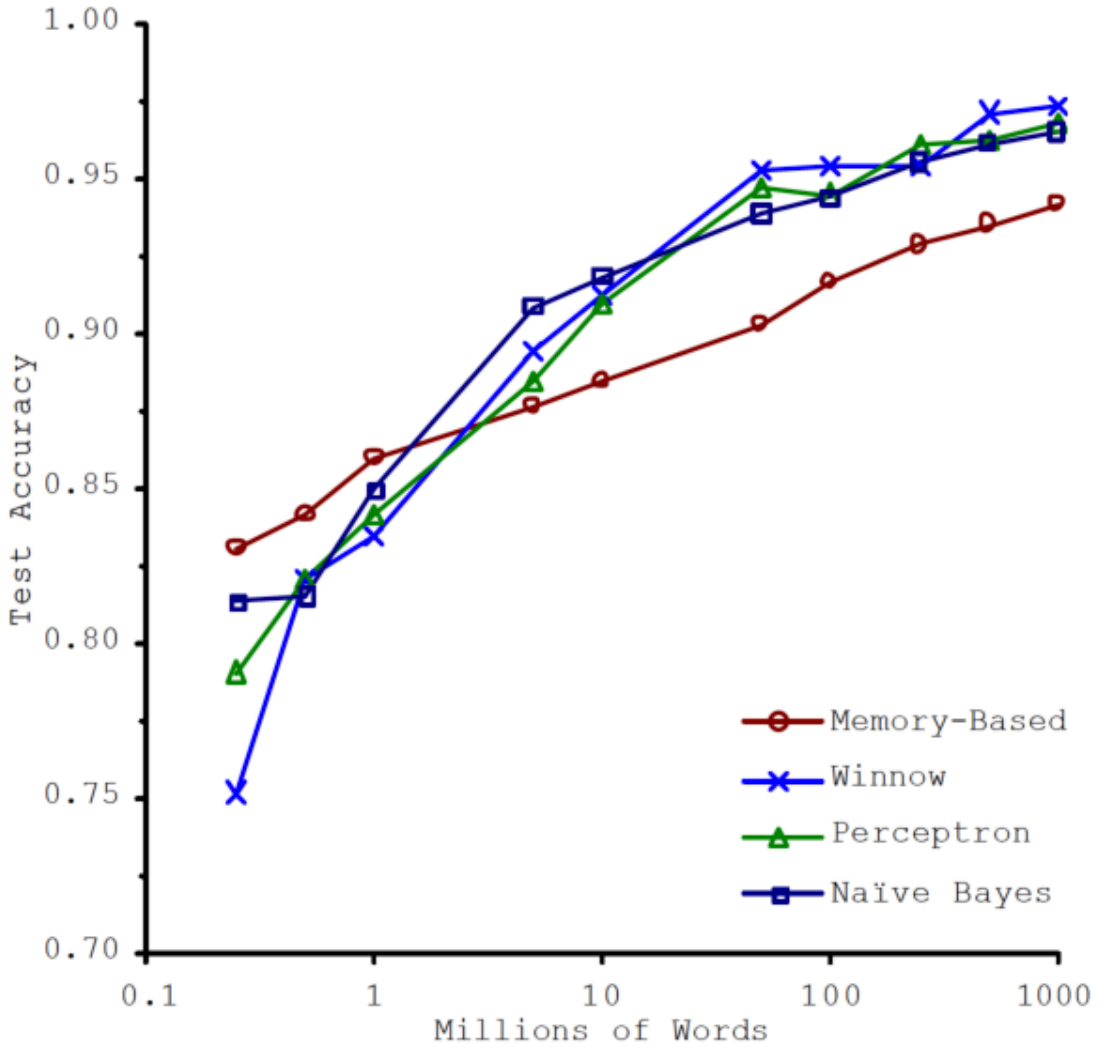
Cette expression est linéaire en $\theta_0$, $\theta_1$ et $\theta_2$, même si elle trace une courbe lorsque $x$ varie. `PolynomialFeatures` change la représentation des attributs; il ne transforme pas l'algorithme d'ajustement en un autre type d'estimateur de régression.

Les coefficients ajustés se rapprochent des coefficients générateurs, car le modèle contient le terme quadratique pertinent et l'échantillon contient un bruit gaussien modéré. Une récupération exacte n'est ni attendue ni nécessaire.

# Quantité et augmentation des données

## La taille compte

Pour la tâche langagière étudiée par Banko et Brill, l'augmentation de la taille du corpus d'entraînement a produit des gains importants pour plusieurs algorithmes d'apprentissage.




**Attribution :** Banko et Brill (2001)

Le résultat de Banko et Brill (2001) porte sur une tâche de désambiguïsation en langage naturel. Il ne faut pas en déduire une loi universelle selon laquelle une plus grande quantité d'exemples résout toujours un problème d'apprentissage.

Des données supplémentaires aident lorsqu'elles apportent de l'information pertinente, correctement étiquetée et représentative. Elles ne corrigent pas une cible inadéquate, des erreurs systématiques d'étiquetage, des sous-populations absentes ou un décalage de distribution entre l'entraînement et le déploiement.

Pour certains grands modèles neuronaux, des lois d'échelle empiriques relient la perte aux données d'entraînement, au nombre de paramètres et au calcul dans un domaine étudié (Kaplan et al. 2020). Ces régularités n'éliminent pas la nécessité d'examiner la qualité et la couverture des données.

## Définition

L'**augmentation des données** (*data augmentation*) crée des exemples d'entraînement supplémentaires au moyen de transformations qui devraient préserver l'étiquette cible.

**But :** encoder des invariances plausibles et améliorer la robustesse aux variations attendues durant le déploiement.

## Augmentation des données

- **Images** : rotations, translations, recadrages, symétries ou bruit, lorsque ces transformations préservent l'étiquette.
  - Comment trouver des géoglyphes anciens à l'aide de l'apprentissage automatique?, Sakai et al. (2024)
- **Texte** : paraphrases ou modifications soigneusement contrôlées qui préservent le sens et l'étiquette attribuée.

Les modèles génératifs peuvent aussi produire des exemples synthétiques, mais une sortie réaliste ne garantit ni des étiquettes correctes, ni une couverture représentative, ni l'indépendance par rapport aux données sources.

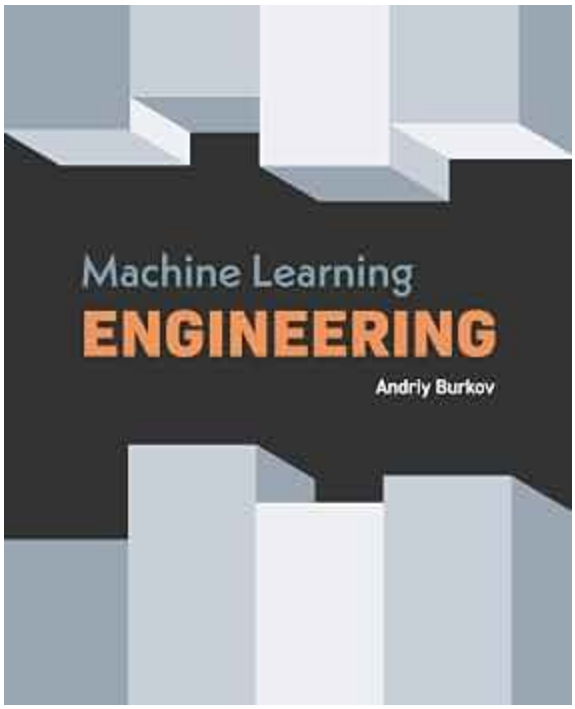
Une augmentation est valide seulement si sa transformation préserve la cible de l'application. Une symétrie horizontale peut être sans conséquence pour plusieurs catégories d'objets, mais invalide pour du texte, la latéralité médicale ou des panneaux de signalisation. Le remplacement de synonymes peut également changer la nuance ou la grammaire.

L'augmentation s'applique uniquement aux exemples d'entraînement et, durant la validation croisée, uniquement au pli d'entraînement. Les ensembles de validation et de test doivent représenter les entrées naturelles.

Les données synthétiques exigent le même examen que les données recueillies. La réutilisation sans discernement de contenu produit par des modèles peut rétrécir la distribution représentée ou amplifier des artefacts. Shumailov et al. (2024) présente une forme de ce risque.

# Conclusion

## Lectures complémentaires




- **Machine Learning Engineering**, par Andriy Burkov (Burkov 2020).
- Traite de la collecte, du stockage et du prétraitement des données, de l'ingénierie des attributs, des tests et du débogage, ainsi que du déploiement, du retrait et de l'entretien des modèles.
- Écrit par l'auteur de **The Hundred-Page Machine Learning Book** (Burkov 2019).
- Offert selon une formule « lire d'abord, acheter ensuite ».

## Résumé

- Faire correspondre l'**encodage catégoriel** au caractère nominal ou ordinal des données.
- Ajuster la **mise à l'échelle**, l'**encodage** et l'**imputation** sur les données d'entraînement au moyen d'un pipeline.
- Choisir la mise à l'échelle selon l'estimateur; elle ne rend pas un attribut normalement distribué.
- Appliquer les **remèdes au déséquilibre des classes** seulement dans les partitions d'entraînement et évaluer sur la distribution naturelle.
- Utiliser des attributs polynomiaux et des interactions pour enrichir la représentation d'un modèle tout en contrôlant sa complexité.
- Privilégier des données représentatives et correctement étiquetées, et n'utiliser que des augmentations qui préservent l'étiquette.

## Prochaine leçon

- **Prochaine étape :** les réseaux de neurones artificiels.

## Références


Banko, Michele, et Eric Brill. 2001. « Scaling to very very large corpora for natural language disambiguation ». *Proceedings of the 39th Annual Meeting on Association for Computational Linguistics* (USA), ACL '01, 26-33. https://doi.org/10.3115/1073012.1073017.

Burkov, A. 2020. *Machine Learning Engineering*. True Positive Incorporated. https://books.google.ca/books?id=HeXizQEACAAJ.

Burkov, Andriy. 2019. *The Hundred-Page Machine Learning Book*. Andriy Burkov.

Chawla, N V, K W Bowyer, L O Hall, et W P Kegelmeyer. 2002. « SMOTE: Synthetic minority over-sampling technique ». *Journal of Artificial Intelligence Research* 16: 321-57.

Géron, Aurélien. 2022. *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3e éd. O'Reilly Media, Inc.

Halevy, Alon, Peter Norvig, et Fernando Pereira. 2009. « The Unreasonable Effectiveness of Data ». *IEEE Intelligent Systems* 24 (2): 8-12.

Kaplan, Jared, Sam McCandlish, Tom Henighan, et al. 2020. *Scaling Laws for Neural Language Models*. https://arxiv.org/abs/2001.08361.

Sakai, Masato, Akihisa Sakurai, Siyuan Lu, et al. 2024. « AI-accelerated Nazca survey nearly doubles the number of known figurative geoglyphs and sheds light on their purpose ». *Proceedings of the National Academy of Sciences* 121 (40): e2407652121. https://doi.org/10.1073/pnas.2407652121.

Shumailov, Ilia, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, et Yarin Gal. 2024. « AI models collapse when trained on recursively generated data ». *Nature* 631 (8022): 755-59. https://doi.org/10.1038/s41586-024-07566-y.


# Annexe : vidéo

## Unreasonable Effectiveness of Data

https://youtu.be/yvDCzhbjYWs

Présentation complémentaire liée à Halevy et al. (2009) et à des travaux empiriques ultérieurs sur les lois d'échelle, comme Kaplan et al. (2020).

La présentation de Peter Norvig dure un peu plus d'une heure. Elle est fournie pour l'étude autonome plutôt que pour un visionnement en classe. L'article de 2009 qui l'accompagne précède la vague moderne des grands systèmes d'apprentissage profond, mais demeure utile pour examiner la relation entre l'échelle des données et les choix de modélisation.

# Annexe : pipeline

## Pipeline

In [35]:
```
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median", add_indicator=True)),
    ('scaler', StandardScaler()),
])

preprocessor = ColumnTransformer([
    (
        "numeric",
        numeric_pipeline,
        make_column_selector(dtype_include=np.number),
    ),
    (
        "categorical",
        OneHotEncoder(handle_unknown="ignore"),
        make_column_selector(dtype_exclude=np.number),
    ),
])

full_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression()),
])
```

Ce modèle garde chaque étape apprise du prétraitement avec l'estimateur. Les colonnes numériques sont imputées et standardisées, tandis que les colonnes catégorielles sont soumises à un encodage one-hot. Les sélecteurs choisissent les colonnes selon leur type, de sorte que le modèle ne dépend pas des noms utilisés dans cette leçon.

L'appel à `full_pipeline.fit(X_train, y_train)` ajuste chaque composante sur les mêmes exemples d'entraînement. Le pipeline ajusté peut ensuite transformer et classifier de façon cohérente les exemples de validation, de test ou de production.

---

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

École de science informatique et de **génie électrique** (**ÉSIGÉ**)

Université d'Ottawa